

ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям
ПРЕПОДАВАТЕЛЬ	Астафьев Рустам Уралович
СЕМЕСТР	2 семестр, 2024/2025 уч. год

Практическое занятие №9. Контрольная работа №3. Форма с валидацией данных.....	2
Контрольная работа №3.....	2
Практическая часть №1.....	2
Задание.....	7
Практическая часть №2.....	9
Задание.....	13

Практическое занятие №9. Контрольная работа

№3. Форма с валидацией данных

Контрольная работа №3

Контрольная работа №3 представляет из себя ответы на вопросы по программе, разработанной в рамках практического занятия №8. В рамках «защиты» необходимо:

- продемонстрировать работоспособность программы;
- ответить на содержательный вопрос по программе (какие элементы используются и как функционируют);
- ответить на вопрос по работе программы (объяснение работы фрагментов кода).

Защиту принимает преподаватель практического занятия в соответствии с установленным им регламентом.

Практическая часть №1

Для работы со строками в Qt реализован класс QString, объекты которого позволяют хранить строки в формате Unicode. Практически вся функциональность в Qt, связанная со строками использует QString.

Класс QString по своей сути, аналогичен «вектору», т.е. QString – это контейнер, хранящий элементы символьного типа QChar. Таким образом, QString предоставляет большое количество методов и операторов для работы с символами, строками и подстроками.

Строки можно сравнивать друг с другом при помощи операторов сравнения ==, !=, <, >, <= и >=, например:

```
QString str = "Qt";
```

```
bool b1 = (str == "Qt"); // b1 = true bool b2 = (str == "QT"); // b2 = false
```

Как видно, результат сравнения зависит от регистра символов. При помощи метода isEmpty() можно узнать, не пуста ли строка:

```
QString str;
```

```
bool b1 = str.isEmpty(); // true
```

Того же результата можно добиться, проверив длину строки методом `length()`:

```
QString str;
```

```
bool b1 = (str.length() == 0); // true
```

В классе `QString` имеются различия между пустыми и нулевыми строками, таким образом, строка, созданная при помощи конструктора по умолчанию, является нулевой строкой. Например:

```
QString str1 = ""; QString str2; str1.isNull(); // false str2.isNull(); // true
```

Объединение можно произвести разными способами, например, при помощи операторов `+=` и `+` или вызовом метода `append()`. Например:

```
QString str1 = "Библиотека"; QString str2 = "Qt";
```

```
QString str3 = str1 + str2; // str3 = "Библиотека Qt" str1.append(str2);  
//str1 = "Библиотека Qt"
```

Для замены определенной части строки можно использовать метод `replace()`. Например:

```
QString str = "Программирование"; str.replace("ирование", "а"); // str =  
"Программа"
```

Для преобразования строки в верхний или нижний регистр используются методы `toLowerCase()` или `toUpperCase()`. Например:

```
QString str1 = "ПрОгРаМмА";
```

```
QString str2 = str1.toLowerCase(); // str2 = "программа" QString str3 =  
str1.toUpperCase(); // str3 = "ПРОГРАММА"
```

Строка может быть разбита на массив строк при помощи метода `split()`. Результатом будет объект `QStringList`, являющийся контейнером, хранящим объекты `QString`.

Таким образом можно разделить строку, содержащую предложение на отдельные слова. В качестве аргумента указывается символ, который является разделителем, в данном случае это пробел.

```
QString str = "Библиотека Qt";  
QStringList list = str.split(" "); // "Библиотека", "Qt"
```

Рассмотрим также другой случай:

```
QString str = "Библиотека Qt ";  
QStringList list = str.split(" "); // "Библиотека", "Qt", ""
```

В данном случае после "Библиотека Qt" был добавлен пробел, так что получилось выражение "Библиотека Qt ", поэтому после выполнения метода split(), было получено 3 элемента, последний из которых пустая строка.

Чтобы избежать этого, необходимо указать флаг, исключающий пустые строки:

```
QString str = "Библиотека Qt "; QStringList list =  
str.split(" ", Qt::SkipEmptyParts); // "Библиотека", "Qt"
```

Таким образом были получены все слова, без пустот.

Операция объединения списка строк в одну строку производится при помощи метода join(). Аргументом указывается символ или строка, который будет вставлен между каждым элементом, при создании единой строки. Например, в данном случае, между словами будет добавлен пробел:

```
QStringList list; list.append("Библиотека"); list.append("Qt");  
str = list.join(" "); // "Библиотека Qt"
```

В строке можно выполнить проверку, что она начинается или заканчивается на определённый символ или подстроку, для этого используются методы startsWith() и endsWith():

```
QString str = "Библиотека Qt";  
bool b1 = str.startsWith("Библ"); // true bool b2 = str.endsWith("Qt");  
// true
```

Если требуется сформировать строку из нескольких значений, можно использовать аргументы. Для этого используется метод arg() с указанием

номера аргумента от 1 до 99, а также в самой строке необходимо указать место, куда будет вставлено значение, с помощью символа %i, где i – номер аргумента.

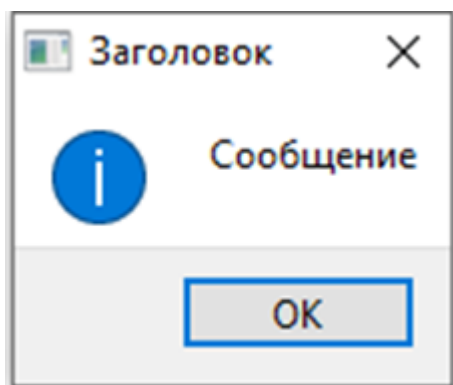
```
QString str = "Дата: %1.%2.%3. %1 %4 шёл дождь.";
str = str.arg(17).arg("09").arg(2024).arg("сентября");
// str = "Дата: 17.09.2024. 17 сентября шёл дождь."
```

Обратите внимание, что при вызове метода arg() значения будут форматироваться в том порядке, в котором они были вызваны, т.е., если в строке указаны аргументы %1 %2 % %3 %1, то при первом вызове метода arg() будут заменены все значения %1, при втором – все %2, а при третьем – все %3.

Применять форматирование строк можно в различных случаях, например, для вывода строки сообщения пользователю, можно воспользоваться классом QMessageBox.

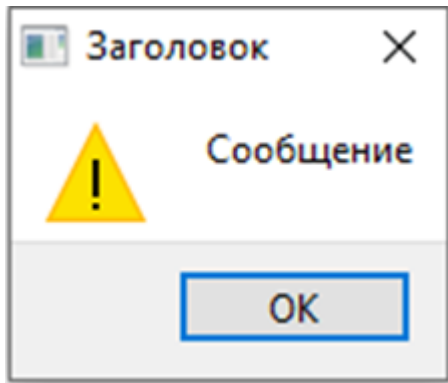
Подключить его можно директивой #include <QMessageBox> А для вывода сообщений доступны несколько вариантов: Информативное сообщение:

```
QMessageBox::information(this, "Заголовок", "Сообщение");
```



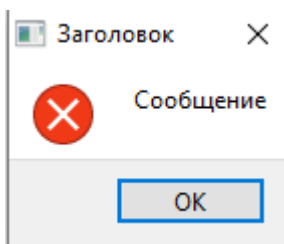
Сообщение с предупреждением:

```
QMessageBox::warning(this, "Заголовок", "Сообщение");
```



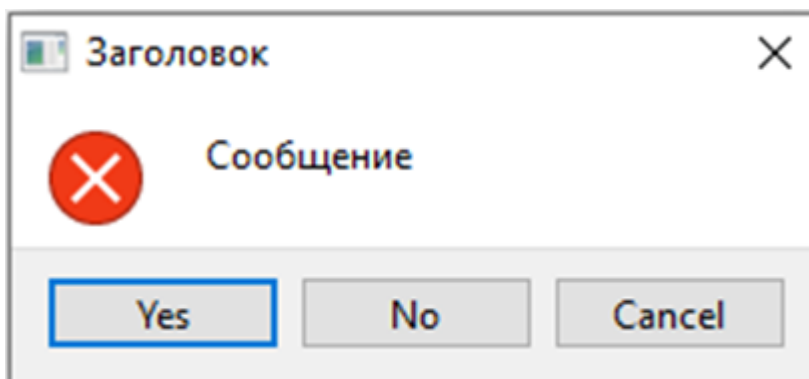
Сообщение о критической ошибке:

```
QMessageBox::critical(this, "Заголовок", "Сообщение");
```



Также в диалоговое окно можно добавить кнопки и проверить, какая из них была нажата пользователем:

```
QMessageBox::StandardButton chose = QMessageBox::critical(this,  
"Заголовок",  
"Сообщение",  
QMessageBox::Yes|QMessageBox::No|QMessageBox::Cancel); if(chose ==  
QMessageBox::Yes)  
{  
    // выбрана кнопка "Да"  
}
```



Таким образом можно выводить информацию для пользователя с помощью диалоговых окон.

Задание

Создать программу с формой регистрации, в которой должны присутствовать следующие параметры:

1. Имя пользователя, не более 15 символов латиницы, допускается использование цифр.
2. Строка для ввода фамилии, имени и отчества в одну строку, разделенную пробелами. Проверить, что ФИО начинается с заглавной буквы, содержит только буквы. Длина слова (фамилии, имени и отчества) не должна превышать 15 символов.
3. Переключатель выбора пола человека.
4. Строка паспорта в формате «серия <пробел> номер», например, 4211 324521.
5. Строка даты рождения в виде строки в формате «день.месяц.год», например, 01.08.2005 – наличие 0 в числах до 10 обязательно.
6. Строка номера телефона в формате +7-XXX-XXX-XX-XX.
7. Строка e-mail, в правильном формате, длина e-mail не более 20 символов.

Для каждого поля выполнить проверку корректности данных на основе регулярных выражений (следующий раздел). В случае корректности заполнения всех полей вывести пользователю информационное сообщение следующего формата:

«Вы успешно зарегистрировали аккаунт <имя пользователя>. Ваше имя:

<имя>, ваша фамилия: <фамилия>, ваше отчество: <отчество>. Ваш пол: <пол текстом>. Серия Вашего паспорта: <серия паспорта>, номер:

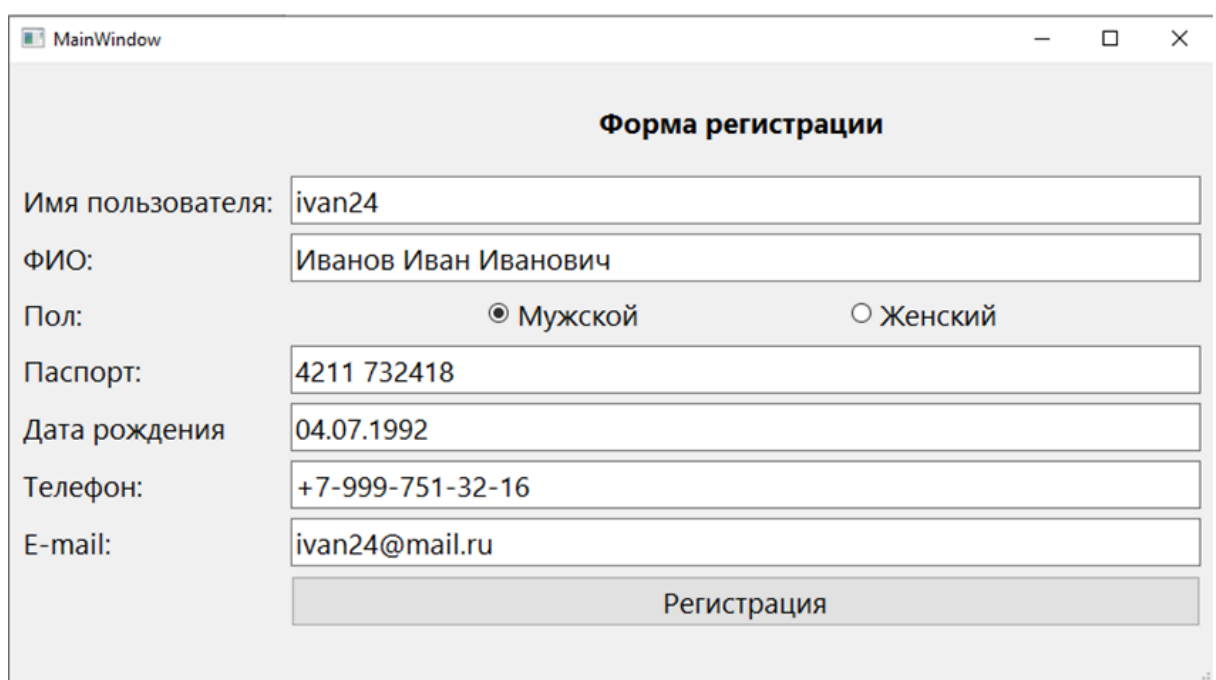
<номер паспорта>. Вы родились <дата, месяц текстом, год>. Ваш номер телефона:

<номер телефона начиная с 8, без пробелов>. Ваш e-mail: <e-mail>. Спасибо за регистрацию!».

Вместо <...> необходимо вставить корректные данные с формы.

В случае, если введенные данные некорректные – отобразить сообщение об ошибке, с указанием проблемы.

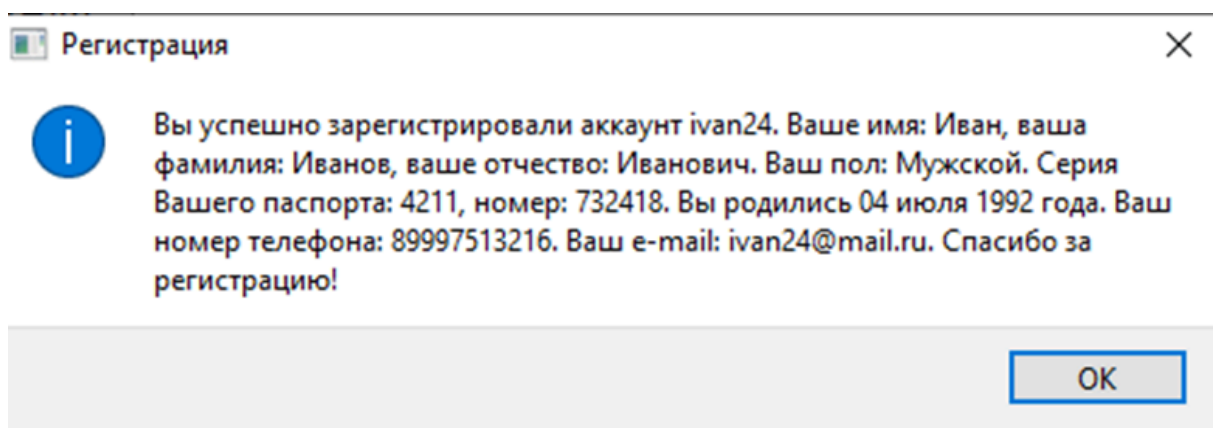
Пример формы:



The screenshot shows a Windows-style window titled "MainWindow" containing a registration form titled "Форма регистрации". The form has the following fields and controls:

- Имя пользователя:
- ФИО:
- Пол: ☒ Мужской ☐ Женский
- Паспорт:
- Дата рождения:
- Телефон:
- E-mail:
- At the bottom, there is a wide button labeled "Регистрация".

Пример сообщения:

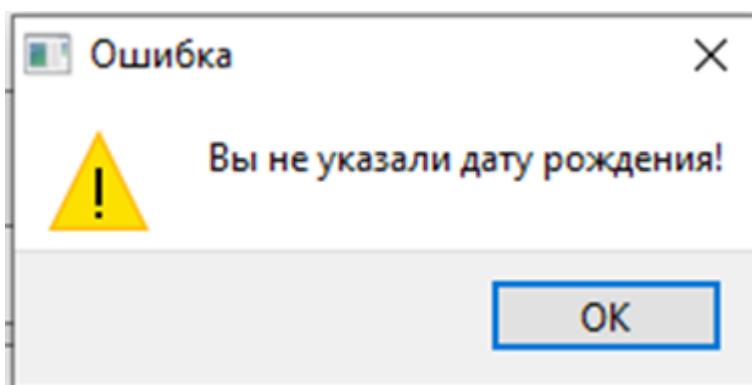


The screenshot shows a small dialog box titled "Регистрация" with a close button (X) in the top right corner. It contains an information icon (i) and a text message:

Вы успешно зарегистрировали аккаунт ivan24. Ваше имя: Иван, ваша фамилия: Иванов, ваше отчество: Иванович. Ваш пол: Мужской. Серия Вашего паспорта: 4211, номер: 732418. Вы родились 04 июля 1992 года. Ваш номер телефона: 89997513216. Ваш e-mail: ivan24@mail.ru. Спасибо за регистрацию!

At the bottom right of the dialog is an "OK" button.

Пример ошибки:



Практическая часть №2

Регулярное выражение – это последовательность символов, представляющая из себя шаблон выражения (текста), позволяющий находить подстроки в тексте. Обычно используется для поиска и замены подстроки в тексте.

Для использования регулярного выражения необходимо задать паттерн (шаблон), по которому будет производиться поиск. Шаблоны регулярных выражений представлены в таблице:

Символ	Описание	Пример
.	Любой символ	a.b
\$	Должен быть конец строки	Abc\$
[]	Любой символ из заданного набора	[abc]
-	Определяет диапазон символов в группе []	[0-9A-Za-z]
^	В начале набора символов означает любой символ, не вошедший в набор	[^def]
*	Символ должен встретиться в строке ни разу или несколько раз	A*b
+	Символ должен встретиться в строке минимум 1 раз	A+b
?	Символ должен встретиться в строке 1 раз или не встретиться вообще	A?b

{n}	Символ должен встретиться в строке указанное число раз	A{3}b
-----	--	-------

{n,}	Допускается минимум n совпадений	a{3,}b
{,n}	Допускается до n совпадений	a{,3}b
{n,m}	Допускается от n до m совпадений	a{2,3}b
	Ищет один из двух символов	ac bc
\b	В этом месте присутствует граница слова	a\b
\B	Границы слова нет в этом месте	a\Bd
()	Ищет и сохраняет в памяти группу найденных символов	(ab ac)ad
\d	Любое число	
\D	Все, кроме числа	
\s	Любой тип пробелов	
\S	Все, кроме пробелов	
\w	Любая буква, цифра или знак подчеркивания	
\W	Все, кроме букв	
\A	Начало строки	
\b	Целое слово	
\B	Не слово	
\Z	Конец строки (совпадает с символом конца строки или перед символом перевода каретки)	
\z	Конец строки (совпадает только с концом строки)	

Для того чтобы найти конкретные символы, нужно поместить их в квадратные скобки, например, [ab] будет искать совпадение с a или b. Чтобы не писать все символы алфавита подряд, можно указать диапазон, например [A-Z] совпадает с любой буквой в верхнем регистре, [a-z] — с любой буквой в нижнем регистре, а [0-9] — с любой цифрой. Можно

совмещать такие записи, например, [a-z7] будет совпадать с любой буквой в нижнем регистре и с цифрой 7.

Также можно исключать символы, поставив перед ними знак ^.

Например, [^0-9] будет соответствовать всем символам, кроме цифр.

В фигурных скобках указываются величины, называемые пределами. Они позволяют точно задать количество раз, которое символ должен повторяться в тексте. Например, a{4,5} будет совпадать с текстом, если буква a встречается в нем от 4 до 5 раз подряд. Например, в следующем отрывке задано регулярное выражение для ip-адреса, с помощью которого можно проверить строку на содержание в ней ip-адреса:

```
QRegularExpression  
reg("[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}"); QString str = "Эта  
строка содержит ip-адрес 123.222.63.1";  
Bool b1 = str.contains(reg);    //true
```

Стоит отметить, что для указания символа точки в регулярном выражении перед ним стоит обратная косая черта (\), а в соответствии с правилами языка программирования C++ для ее задания в строке она должна удваиваться, тем самым «экранируя» спецсимвол. Если бы косой черты не было, то точка имела бы в соответствии с таблицей значение «любой символ», и регулярное выражение распознало бы, например, строку “1z2y3x4”, как ip-адрес, что, разумеется, не правильно.

Шаблоны можно комбинировать при помощи символа |, задавая ветвления в регулярном выражении – также, как при использовании операторов if/else в коде. Регулярное выражение с двумя ветвями совпадает с подстрокой, если совпадает одна из ветвей. Например:

```
QRegularExpression rxp("(com|.ru)");  
bool b1 = rxp.match\("www.bhv.ru"\).hasMatch\(\);    // true bool b2 =  
rxp.match\("www.bhv.de"\).hasMatch\(\);    // false
```

Указанные в таблице символы с обратной косой чертой (обратным слешем) позволяют значительно упростить регулярные выражения. Например, регулярное выражение `[a-zA-Z0-9_]` идентично выражению `\w`.

В Qt для работы с регулярными выражениями используется класс `QRegularExpression`.

Для выполнения простого сопоставления, можно вызвать метод `match()`:

```
// сопоставление двух цифр, пробела и слово QRegularExpression
re("\\d\\d \\w+"); QRegularExpressionMatch match = re.match("abc123 def");
bool hasMatch = match.hasMatch(); // true
```

Если сопоставление успешно, с помощью метода `.captured(0)` можно извлечь выделенную подстроку:

```
QRegularExpression re("\\d\\d \\w+"); QRegularExpressionMatch match
= re.match("abc123 def"); if (match.hasMatch()) {
    QString matched = match.captured(0); // matched == "23 def"
    // ...
}
```

В данном случае аргумент `0` означает, что мы хотим получить подстроку, соответствующую всему шаблону.

Также возможно извлечение конкретных подстрок, начиная с `1`, если в шаблоне используются группы:

```
QRegularExpression re("(\\d\\d)/(\\d\\d)/(\\d\\d\\d\\d\\d)$");
QRegularExpressionMatch match = re.match("08/12/1985");
if (match.hasMatch()) {
    QString day = match.captured(1); // day == "08"
    QString month = match.captured(2); // month == "12"
    QString year = match.captured(3); // year == "1985"
    // ...
}
```

Обратите внимание, что группы нумеруются начиная с 1.

Также в Qt имеется перегрузка этой функции, требующая QString как параметр для извлечения именованных подстрок:

```
QRegularExpression
re("^(?<date>\\d\\d)/(?<month>\\d\\d)/(?<year>\\d\\d\\d\\d)$");
QRegularExpressionMatch match = re.match("08/12/1985");
if (match.hasMatch()) {
    QString date = match.captured("date"); // date == "08"
    QString month = match.captured("month"); // month == "12"
    QString year = match.captured("year"); // year == 1985
}
```

Таким образом возможно использовать строковые обозначения при извлечении подстрок.

Задание

Доработать программу из предыдущей практической работы, заменив проверку вводимых данных с помощью регулярных выражений:

1. Имя пользователя, не более 15 символов латиницы, допускается использование цифр.
2. Строка для ввода фамилии, имени и отчества в одну строку, разделённую пробелами. ФИО должны начинаться с заглавной буквы, содержит только буквы. Длина слова (фамилии, имени и отчества) не должна превышать 15 символов.
3. Строка паспорта в формате «серия <пробел> номер», например,
4211 324521.
4. Строка даты рождения в виде строки в формате «день.месяц.год», например, 01.08.2005 – наличие 0 в числах до 10 обязательно.
5. Строка номера телефона в формате +7-XXX-XXX-XX-XX.

6. Строка e-mail, в правильном формате, длина e-mail не более 20 СИМВОЛОВ.